

ELECTRONICS GUIDE | جزوهٔ آموزشی مهندسی الکترونیک

واحد جانبی PIOC
در میکروکنترلر CH32X035
از ساختار اجرایی تا راه‌اندازی نوار LED تک‌سیمهٔ WS2812

از مرور ساختار اجرایی PIOC و سه حلقهٔ تعامل با میزبان (Host) — بارگذاری، ارسال فرمان، خواندن نتیجه — و اصول طراحی زمان‌بندی (Timing) بر پایهٔ چرخهٔ دستور، تا مثال عملی راه‌اندازی نوار LED تک‌سیمهٔ WS2812 با بارگذاری برنامه، واکشی داده با دستورات اسمبلی (Assembly)، ارسال بیت‌به‌بیت و پردازش نتیجه

در این جزوه، ترجمهٔ فارسی سند رسمی AN23000 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را می‌خوانید: سری‌های CH32X035، CH32H417 و CH32V205 به یک واحد جانبی (Peripheral) قابل‌برنامه‌ریزی به‌نام PIOC (همان eMCU) مجهزند که با هستهٔ RISC8B و فضای برنامه و رجیسترهای مستقل، بدون اشغال وقفه و منابع زمان‌بندی میزبان روی دو پایه موج پروتکل تولید می‌کند. دو شیوهٔ انتقال داده (SFR و منطقهٔ برنامه با RDCODE)، مدولاسیون PWM و Manchester، تایمر ۰ و دستور انتظار WAITB گام‌به‌گام مرور می‌شود.

میکروکنترلر CH32X035 · واحد جانبی قابل‌برنامه‌ریزی PIOC · هستهٔ RISC8B و اسمبلی
پروتکل 1-Wire و WS2812 · مدولاسیون PWM / Manchester · تایمر ۰ و طراحی تأخیر
دستور انتظار WAITB · واژه‌نامهٔ فنی

ترجمهٔ فارسی AN23000 V1.0 · **نسخهٔ ۱٫۰** — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

فهرست مطالب

۱ توضیحات و دامنهٔ کاربرد ۳

۲ بخش ۱ | ساختار عملکردی PIOC ۴

1.1 مرور ساختار ۴

1.2 تعامل میزبان (Host) با PIOC ۵

1.3 طراحی زمان‌بندی (Timing) ۷

۳ بخش ۲ | نمونهٔ راه‌اندازی نوار LED تک‌سیمهٔ WS2812 ..... ۱۲

2.1 پنجره‌های زمانی پروتکل ۱۲

2.2 مؤلفه‌های پروژه و مقداردهی اولیه ۱۳

2.3 کدهای فرمان و کدهای نتیجه ۱۳

2.4 مسیرهای فریم کوتاه و فریم بلند ۱۴

2.5 فراخوانی میزبان و یک ارسال کامل ۱۷

۴ تاریخچهٔ نسخه‌ها و بیانیه ۱۸

۵ واژگان کلیدی (واژه‌نامهٔ فنی) ۱۸

فهرست شکل‌ها، جدول‌ها و کدها

شکل ۱-۱ ساختار PIOC در CH32X035 ..... ۴

شکل ۱-۲ سه بیت وضعیت فرمان و نتیجه ۵

شکل ۱-۳ دو شیوهٔ انتقال داده ۶

شکل ۱-۴ تخصیص نرم‌افزاری دو بافر SFR ۷

شکل ۱-۵ اشغال دستورها و فاصلهٔ لبه‌ها ۸

شکل ۱-۶ دو نوع مدولاسیون بیت در PIOC ۹

شکل ۱-۷ شمارش در حالت تایمر ۱۰

شکل ۱-۸ شمارش در حالت PWM ۱۱

شکل ۲-۱ تعریف بیت WS2812 ..... ۱۲

شکل ۲-۲ سه مسیر در کد ۱۴

شکل ۲-۳ زمان‌بندی بیت نرم‌افزاری ۱۵

جدول ۱-۱ ویژگی‌های اصلی PIOC ۵

جدول ۱-۲ فایل‌های اصلی پروژهٔ نمونه ۱۳

جدول ۲-۲ تخصیص کدهای فرمان در مثال ۱۴

جدول ۲-۳ ترتیب عملیات دو رابط ارسال میزبان ۱۷

چکیده —

میکروکنترلرهای سری CH32X035، CH32H417، CH32V205 و برخی دیگر از محصولات MCU، به یک واحد جانبی (Peripheral) با نام PIOC مجهز شده‌اند.

در این مقاله، با استفاده از میکروکنترلر CH32X035 و پروژه نمونه 1_Wire، ساختار اجرایی PIOC، نحوه تبادل داده با واحد میزبان (Host) و طراحی زمان‌بندی (Timing) آن معرفی می‌شود.

همچنین، با استفاده از راه‌اندازی و کنترل نوار LED تک‌سیمه WS2812، روش‌های بارگذاری برنامه، دریافت داده با استفاده از دستورات اسمبلی (Assembly)، ارسال داده به‌صورت بیت‌به‌بیت و پردازش نتایج بررسی خواهند شد.

مقدمه

توضیحات و دامنهٔ کاربرد

میکروکنترلرهای سری CH32X035، CH32H417، CH32V205 و برخی دیگر از محصولات MCU، به یک واحد جانبی (Peripheral) با نام PIOC مجهز شده‌اند. در این سند، با استفاده از میکروکنترلر CH32X035 و پروژه نمونه 1_Wire، ساختار اجرایی PIOC، نحوهٔ تبادل داده با واحد میزبان (Host) و طراحی زمان‌بندی (Timing) آن معرفی می‌شود. همچنین با راه‌اندازی و کنترل نوار LED تک‌سیمهٔ WS2812، روش‌های بارگذاری برنامه، دریافت داده با استفاده از دستورات اسمبلی (Assembly)، ارسال داده به‌صورت بیت‌به‌بیت و پردازش نتایج بررسی خواهند شد.

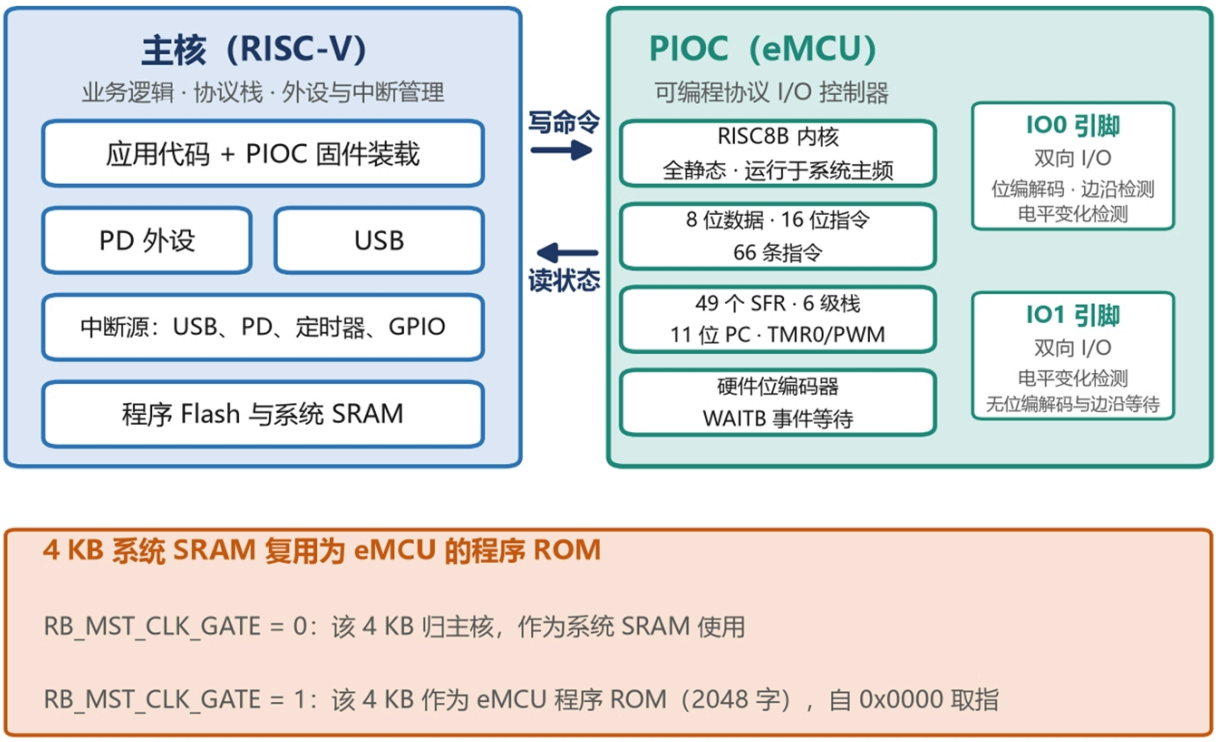
جدول ۱ دامنهٔ کاربرد سند

دامنهٔ کاربرد	سری
میکروکنترلرهای همگانی‌منظوره	CH32X035، CH643، CH32V205، CH32H41x و مانند آن‌ها

بخش ۱

ساختار عملکردی PIOC

◆ ۱.۱ مرور ساختار



شکل ۱-۱ ساختار PIOC در CH32X035

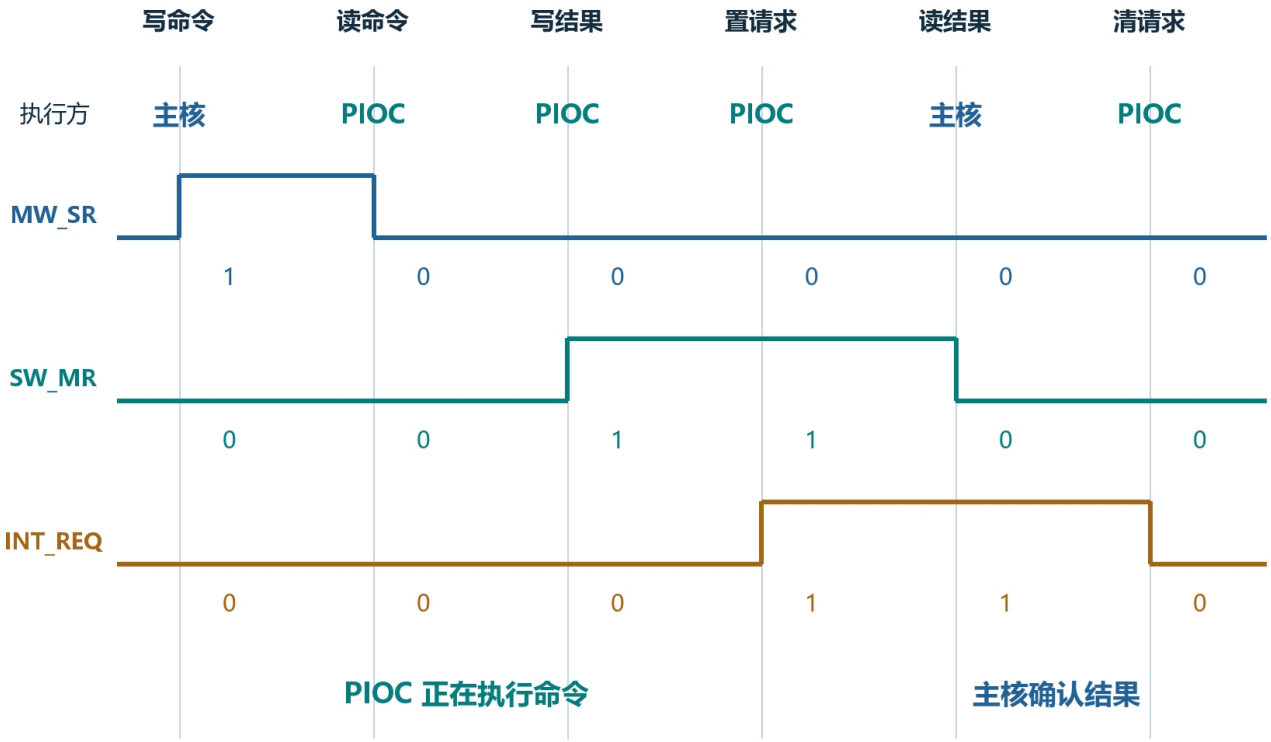
PIOC واحد جانبیِ قابل‌برنامه‌ریزیِ ورودی/خروجی است که درون تراشه تعبیه شده و در راهنمای تراشه با نام eMCU هم شناخته می‌شود. هستهٔ آن RISC8B است و فضای برنامه، بانک رجیسترها و جریان دستورِ مستقل دارد؛ به‌طوری که بدون اشغال وقفه و منابع زمان‌بندی میزبان، می‌تواند با چرخه‌های دستورِ معین روی دو پایه، موج پروتکل تولید کند.

از منظر نحوهٔ استفاده، تعامل میزبان و PIOC در سه حلقهٔ اصلی متمرکز است: بارگذاری (نوشتن دستورهای ترجمه‌شده در SRAM مشترک و سپردن واکشی آن‌ها به PIOC)، محول‌کردن وظیفه (نوشتن فرمان و پارامترها) و خواندن نتیجه (خواندن کد نتیجه پس از پایان تراکنش). پس از تکمیل بارگذاری، اجرای پروتکل به‌طور مستقل توسط PIOC انجام می‌شود و میزبان می‌تواند به کارهای دیگر ادامه دهد. بنابراین جایگاه PIOC را می‌توان چنین خلاصه کرد: یک واحد جانبی ۸بیتیِ قابل‌برنامه‌ریزی توسط کاربر، برای پیاده‌سازی رابط‌هایی با زمان‌بندی معین، نرخ متوسط و تعداد پایهٔ کم. کارکرد آن را برنامهٔ اسمبلیِ نوشتهٔ کاربر تعیین می‌کند، نه ساختار ثابت تراشه.

جدول ۱-۱ ویژگی‌های اصلی PIOC

ویژگی	مشخصه
هسته	RISC8B؛ دادهٔ 8 بیتی، دستور 16 بیتی و 66 دستور.
فرکانس اجرا	هم‌سان با فرکانس اصلی سیستم؛ طراحی تمام‌استاتیک و محاسبهٔ زمان‌بندی بر پایهٔ چرخه.
فضای برنامه	2048 کلمه (4KB)، از طریق اشتراک با SRAM سیستم
رجیسترها	49 عدد SFR (0x00~0x3F) شامل 33 بایت رجیستر داده
پشته	6 سطح با عرض 11 بیت
پایه‌ها	2 مسیر ورودی/خروجی دوسو با پشتیبانی از تشخیص تغییر سطح
کدگذار و کدگشای بیت	PWM (25%/75%) و Manchester؛ عرض بیت قابل‌برنامه‌ریزی
تایمر	تایمر بازبارگذاری فعالِ 8 بیتی T0 با دو حالت تایمر/PWM
شیوهٔ بارگذاری	پشتیبانی از توقف و بارگذاری پویا

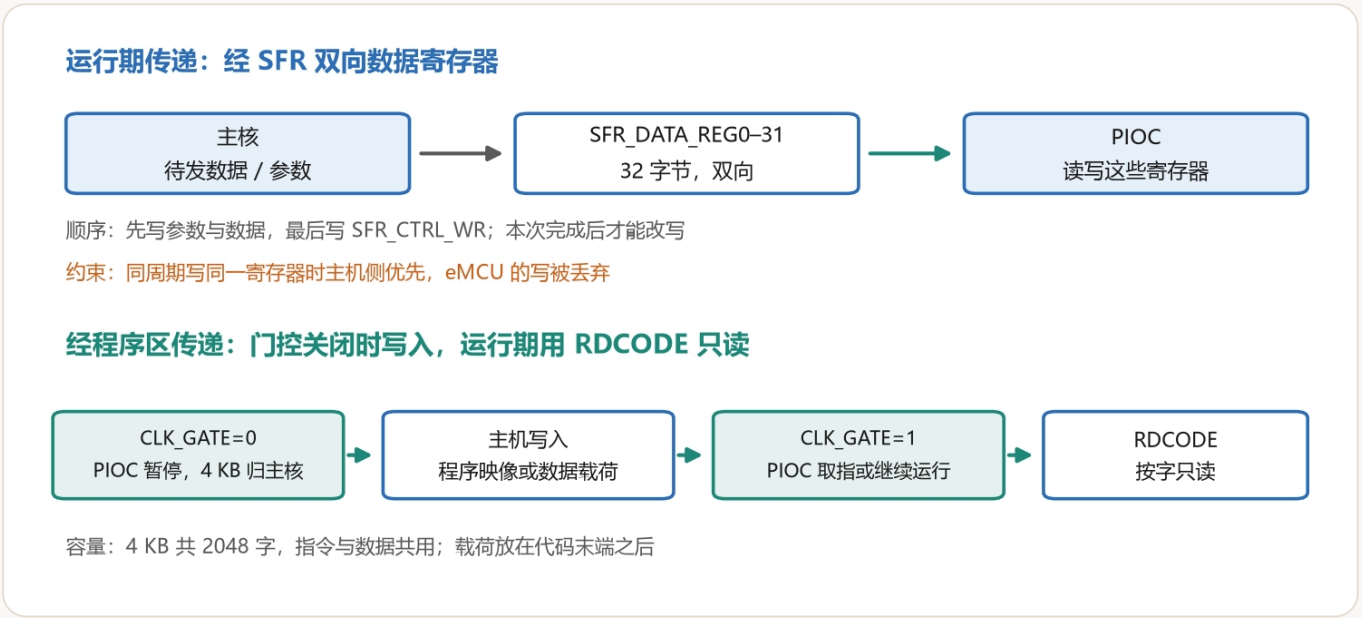
◆ ۱.۲ تعامل میزبان با PIOC



MW_SR 和 SW_MR 随寄存器读写自动变化；本图 INT_REQ 由示例固件控制

شکل ۱-۲ سه بیت وضعیت فرمان و نتیجه

- MW_SR نشان می‌دهد فرمان برداشته شده است یا نه (پس از نوشتن CTRL_WR توسط میزبان، MW_SR به‌طور خودکار یک می‌شود و پس از خواندن CTRL_WR توسط PIOC به‌طور خودکار صفر می‌شود؛ این بیت فقط برداشته‌شدن فرمان را نشان می‌دهد و در آن هنگام پردازش پروتکل تازه آغاز شده است. تفسیر این پرچم تنها همراه با عمل ارسال همان مرحله معنا دارد و نباید مقدار اولیهٔ صفر را نشانهٔ پایان کار گرفت)؛
- SW_MR نشان می‌دهد نتیجه خوانده شده است یا نه (پس از نوشتن CTRL_RD توسط PIOC، SW_MR به‌طور خودکار یک می‌شود و پس از خواندن CTRL_RD توسط میزبان، DATA_SW_MR به‌طور خودکار صفر می‌شود)؛
- INT_REQ توسط PIOC یک می‌شود تا از میزبان درخواست وقفه یا لغوِ درخواست شود و پس از برداشتن نتیجه توسط میزبان، فریم‌ور آن را پاک می‌کند. میزبان می‌تواند با عمل نوشتن جعلی روی رجیستر فقط‌خواندنی CTRL_RD، INT_REQ را صفر کند؛ نوشتن میزبان در CTRL_RD اثر واقعی روی دادهٔ آن ندارد.



شکل ۱-۳ دو شیوهٔ انتقال داده

شیوهٔ SFR برای پارامترها و داده‌های اندک مناسب است. دو طرف پیش‌تر کاربرد رجیسترها را با هم توافق می‌کنند؛ برای مثال DATA0 پیش از ارسال فرمان، توسط میزبان نوشته می‌شود، DATA1 هنگام اجرا توسط PIOC نوشته می‌شود و میزبان پس از رسیدن اعلامِ پایان، آن را می‌خواند. اگر میزبان و PIOC در یک چرخه روی یک رجیستر مشترک بنویسند، نوشتن میزبان مقدم می‌شود و نوشتن PIOC دور ریخته می‌شود.

شیوهٔ منطقهٔ برنامه برای داده‌های فقط‌خواندنیِ بیشتر یا جدول ثابت‌ها مناسب است. این 4KB در CLK_GATE=0 در اختیار میزبان است و در CLK_GATE=1 به‌عنوان ROM برنامهٔ PIOC عمل می‌کند؛ بنابراین نوشتن باید در فاصلهٔ بسته‌بودن گیت (یا حفظ حالت ریست) انجام شود و پس از پایان نوشتن، CLK_GATE یک شود. PIOC هنگام خواندن از RDCODE استفاده می‌کند: 8 بیت پایین آدرس هدف در SFR_INDIR_ADDR و 3 بیت بالا در رجیستر A نوشته می‌شود و یک بار خواندن، 16 بیت داده برمی‌گرداند؛ آدرس‌دهی بر پایهٔ آدرس کلمهٔ 11 بیتی است.

منطقهٔ برنامه در مجموع 2048 کلمه است و دستور و داده در آن مشترک‌اند؛ PIOC فقط اجازهٔ خواندن دارد و اجازهٔ نوشتن ندارد و نتیجه‌ای که تولید می‌کند تنها از مسیر SFR به میزبان بازمی‌گردد. در طول CLK_GATE=1 میزبان نیز اجازهٔ نوشتن در این ناحیه را ندارد و برای به‌روزرسانی داده‌ها باید دوباره گیت را ببندد.



شکل ۱-۴ تخصیص نرم‌افزاری دو بافر SFR

وقتی داده در یک نوبت جا نمی‌شود، می‌توان رجیسترهای DATA_REG موجود را به دو بلوک تقسیم کرد. هنگامی که PIOC بلوک A را می‌خواند، میزبان فقط بلوک B را می‌نویسد؛ پیش از هر جابه‌جایی باید هم‌زمان دو شرط «A خوانده شده است» و «B آماده است» برقرار باشند. در جهت دریافت، وضعیت برعکس است: PIOC در بلوک خالی می‌نویسد و میزبان بلوک کامل‌شده را می‌خواند. طول بلوک، پرچم اعتبار، طول آخرین بلوک و مدیریت مهلت همه باید توسط نرم‌افزار تعریف شوند. در نمونهٔ EVT، مثال‌های UART و IIC اعلان دادهٔ بخش‌بخش را شامل می‌شوند و می‌توان به تخصیص رجیسترها و شیوهٔ شمارش آن‌ها مراجعه کرد.

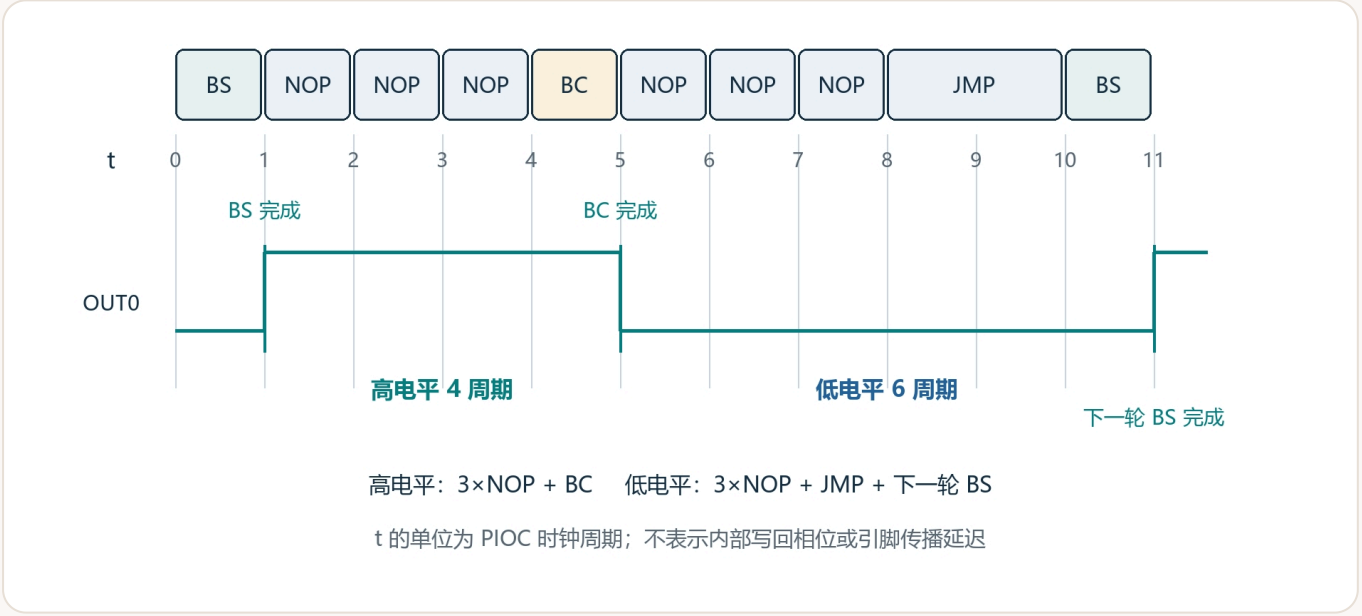
◆ ۱.۳ طراحی زمان‌بندی

PIOC بر هستهٔ RISC8B با مجموعهٔ دستور ساده‌شدهٔ تک‌چرخه‌ای تکیه دارد، با فرکانس اصلی سیستم اجرا می‌شود و طراحی زمان‌بندی آن بر واحد چرخهٔ دستور استوار است. آنچه راهنمای پروتکل معمولاً ارائه می‌دهد نانوثانیه یا میکروثانیه است که باید نخست به تعداد چرخه تبدیل شود. PIOC دستور تأخیر اختصاصی ندارد و تأخیر از اجرای عمل‌های تهی ساخته می‌شود و بنا بر طول، در سه رده قرار می‌گیرد: برای 1~10 چرخه می‌توان از NOPهای پیاپی بهره گرفت؛ برای 10~100 چرخه می‌توان زیربرنامه‌ای حاوی n دستور NOP را فراخوانی کرد (تعداد چرخه‌ها n+4 است و CALL و RET هر کدام 2 چرخه)؛ برای چرخه‌های بلندتر می‌توان از حلقهٔ شمارشی استفاده کرد که در هر دور، DEC و JNZ روی هم 3 چرخه می‌برند.

برای تبدیل یک قطعه کد به شکل موج، تنها باید چرخه‌ها را در طول مسیر اجرا جمع کرد. در مثال زیر فرض شده IO0 به‌عنوان خروجی نرم‌افزاری پیکربندی شده و کدگذاری بیت و خروجی تایمر هر دو بسته‌اند:

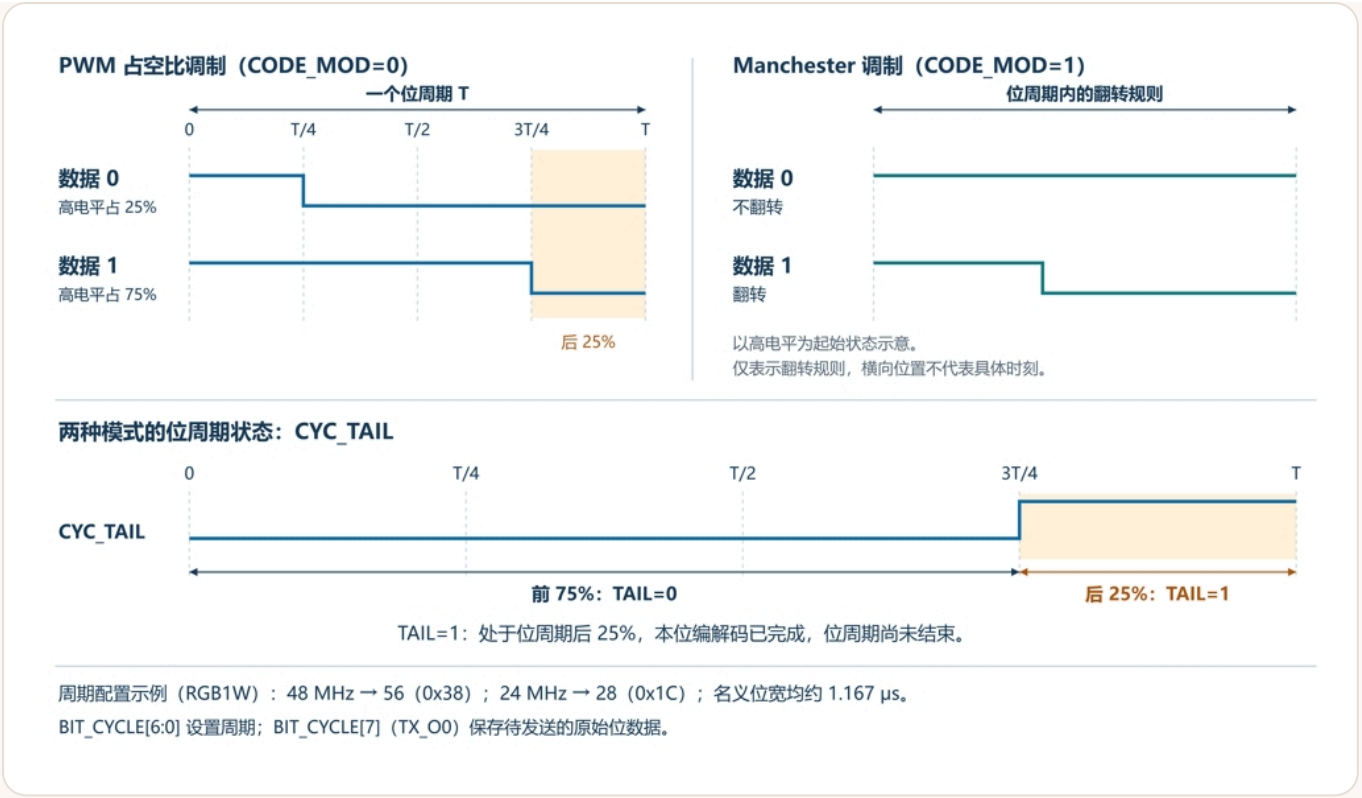
```
PULSE:  BS    SFR_PORT_IO,SB_PORT_OUT0
        NOP
        NOP
        NOP
        BC    SFR_PORT_IO,SB_PORT_OUT0
        NOP
        NOP
        NOP
        JMP   PULSE
```

در شکل، تغییر مقدار قفل خروجی بر مبنای مرزِ پایان هر دستور نشان داده شده است: BS اول در t=1 کامل می‌شود، BC در t=5 و BS دور بعد در t=11. سطح بالا 4 چرخه و سطح پایین 6 چرخه است؛ در 48MHz هر چرخه حدود 20.833ns است و سطوح اسمیِ بالا و پایین به‌ترتیب حدود 83.333ns و 125ns می‌شوند. انتقال معمول، عملیات حسابی و عملیات بیتی معمولاً 1 چرخه است؛ JMP، CALL و بازگشت 2 چرخه؛ در پرش شرطی، در صورت برقراری شرط یا رخ‌دادن پرش، چرخه‌های اضافی به حساب می‌آید و RDCODE 3 چرخه است. زمان کل WAITB را زمانِ برقراری رویداد تعیین می‌کند و نمی‌توان زمان انتظار را تأخیر ثابتِ تک‌چرخه‌ای تلقی کرد.



شکل ۱-۵ اشغال دستورها و فاصلهٔ لبه‌ها

کدگذار و کدگشای بیت برای پروتکل‌هایی به کار می‌رود که شکل موج یکسان را بیت‌به‌بیت تکرار می‌کنند: پس از پیکربندی عرض بیت و شیوهٔ کدگذاری، با نوشتن بیت داده در bit7 رجیستر BIT_CYCLE، سخت‌افزار یک دورهٔ بیت کامل تولید می‌کند و CPU می‌تواند هم‌زمان به بیت بعدی رسیدگی کند. در حالت PWM، صفر 25% و یک 75% را در بر می‌گیرد؛ در حالت Manchester، صفر تغییری نمی‌کند و یک در نیمهٔ دوره تغییر حالت می‌دهد؛ TAIL=1 یعنی وارد یک‌چهارم پایانی شده‌ایم و کدگذاری این بیت کامل شده است. نوشتن در BIT_CYCLE خودش دورهٔ بیت را آغاز می‌کند و به بیت آغازگر اضافی نیاز ندارد؛ نوشتن در این رجیستر سجلِ تشخیص لبهٔ IO0 را پاک می‌کند و اگر در همان هنگام با WAITB در انتظار لبه باشید، تحت تأثیر قرار می‌گیرید. با نوشتن صفر در میدان عرض بیت، کدگذاری بسته می‌شود.



شکل ۱-۶ دو نوع مدولاسیون بیت در PIOC

WAITB دستور همگام‌سازی رویداد در PIOC است. تا زمانی که شرط برقرار نشود، شمارندهٔ برنامه بدون تغییر می‌ماند و همان دستور بارها اجرا می‌شود؛ پس از برقراری شرط، اجرا به دستور بعدی ادامه می‌یابد. هشت رویداد، نیازهای اصلی همگام‌سازی را پوشش می‌دهند:

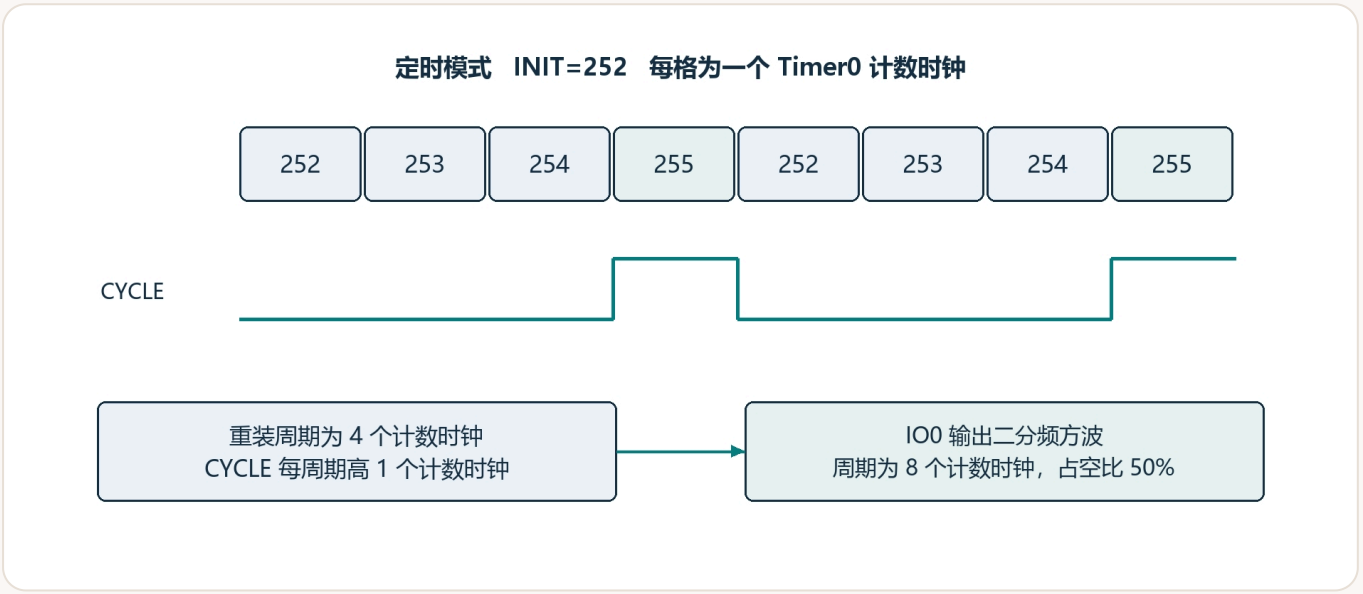
- انتظار برای نوشتن فرمان توسط میزبان (WB_DATA_MW_SR_1)؛
- انتظار برای خواندن نتیجه توسط میزبان (WB_DATA_SW_MR_0)؛
- انتظار برای ورود دورهٔ بیت به یک‌چهارم پایانی و تکمیل کدگذاری بیت (WB_BIT_CYC_TAIL_1)؛
- انتظار برای لبهٔ بالارونده یا پایین‌روندهٔ IO0 (WB_PORT_I0_RISE/FALL)؛
- انتظار برای متفاوت‌شدن یا یکسان‌شدن دوبارهٔ ورودی و خروجی IO0\IO1 (WB_PORT_XOR0_1/0) و (WB_PORT_XOR1_1).

دو نکتهٔ معنایی بر شیوهٔ استفاده اثر می‌گذارند: رویدادهای لبه‌ای هنگام خروج، سجل تشخیص را پاک می‌کنند، بنابراین دو انتظار پیاپی برای لبهٔ بالارونده به‌سبب تشخیص‌شدن لبه در بار قبلی، بی‌درنگ بازنمی‌گردند؛ در مقابل، رویدادهای شرطی اگر از پیش برقرار باشند بی‌درنگ خارج می‌شوند. PORT_XORx نتیجهٔ XOR ورودی با مقدار قفل خروجی را نشان می‌دهد. اگر OUT0=1 باشد، XOR0=1 یعنی ورودی پایین است؛ اگر OUT0=0 باشد، همان XOR0=1 یعنی ورودی بالاست. پس WAITB WB_PORT_XOR0_1 را نمی‌توان همیشه «انتظار برای سطح پایین» تعبیر کرد و نخست باید مقدار مرجع OUT روشن شود.

زمان نمونه‌برداری ورودی همچنین به SB_PORT_IN_EDGE وابسته است: با 0 نمونه‌برداری در میانهٔ چرخهٔ ساعت انجام می‌شود و با 1 مقدار لبهٔ دورهٔ قبلی گرفته می‌شود. هنگام طراحی برنامهٔ دریافت، نخست باید شرط آغاز را پیدا کنید و سپس

نقطهٔ نمونه‌برداری را درون پنجرهٔ مؤثر بگذارید؛ تشخیص لبه به آن معنا نیست که دستور بعدی می‌تواند مرکز بیت را نمونه‌برداری کند.

PIOC یک تایمر 8 بیتی (تایمر 0) در اختیار می‌گذارد: TMR0_COUNT مقدار جاری را نگه می‌دارد، TMR0_INIT مقدار بازبارگذاری I را و TIMER_CTRL تقسیم فرکانس، حالت و خروجی را انتخاب می‌کند. ساعت تایمر از تقسیم فرکانس اصلی سیستم به‌وسیلهٔ SB_TMR0_FREQ به‌دست می‌آید و میدان تقسیم 111~000 به‌ترتیب متناظر با D=1024، 256، 64، 16، 8، 4، 2 و 1 است. تایمر 0 می‌تواند در حالت تایمر یا حالت مدولاتور پهنای پالس کار کند. در حالت تایمر، شمارش از TMR0_INIT تا 0xFF ادامه می‌یابد، سرریز می‌شود و مقدار اولیه بازبارگذاری می‌گردد؛ در حالت PWM دوره بر 256 ساعت تایمر ثابت است و مقدار اولیه به آستانهٔ مقایسه تبدیل می‌شود.



شکل ۱-۷ شمارش در حالت تایمر

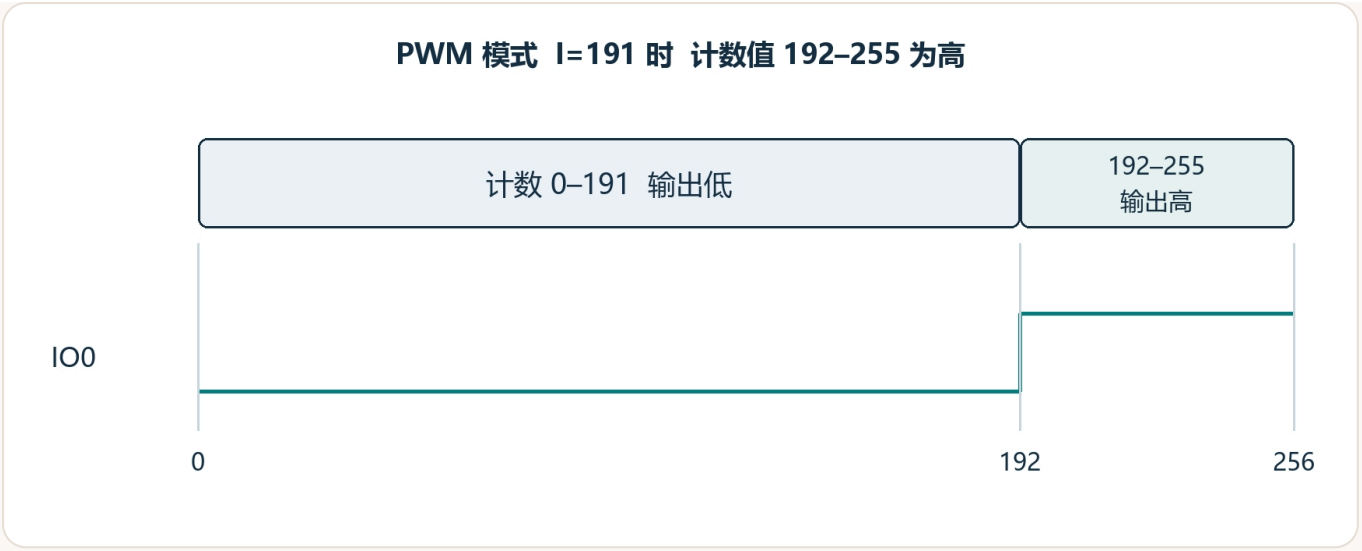
در حالت تایمر، شمارش از I تا 255 ادامه می‌یابد و سپس دوباره I بارگذاری می‌شود. دورهٔ بازبارگذاری برابر است با:

$$T_{reload} = (256 - I) * D / F_{pioc}$$

سیگنال داخلی CYCLE هنگام رسیدن شمارنده به 255 در سطح بالا است و هنگام بازبارگذاری به سطح پایین بازمی‌گردد. پس از اجازهٔ خروجی، IO0 موج مربعیِ حاصل از تقسیم بر دو این سیگنال دوره‌ای را تولید می‌کند؛ فرکانس آن برابر است با:

$$F_{out} = F_{pioc} / [2 * D * (256 - I)]$$

و ضریب مطابق آن 50% است.



شکل ۱-۸ شمارش در حالت PWM

در حالت PWM، شمارش همیشه از 0 تا 255 ادامه می‌یابد و به 0 بازمی‌گردد؛ دوره برابر است با:

$$T_{pwm} = 256 * D / F_{pioc}$$

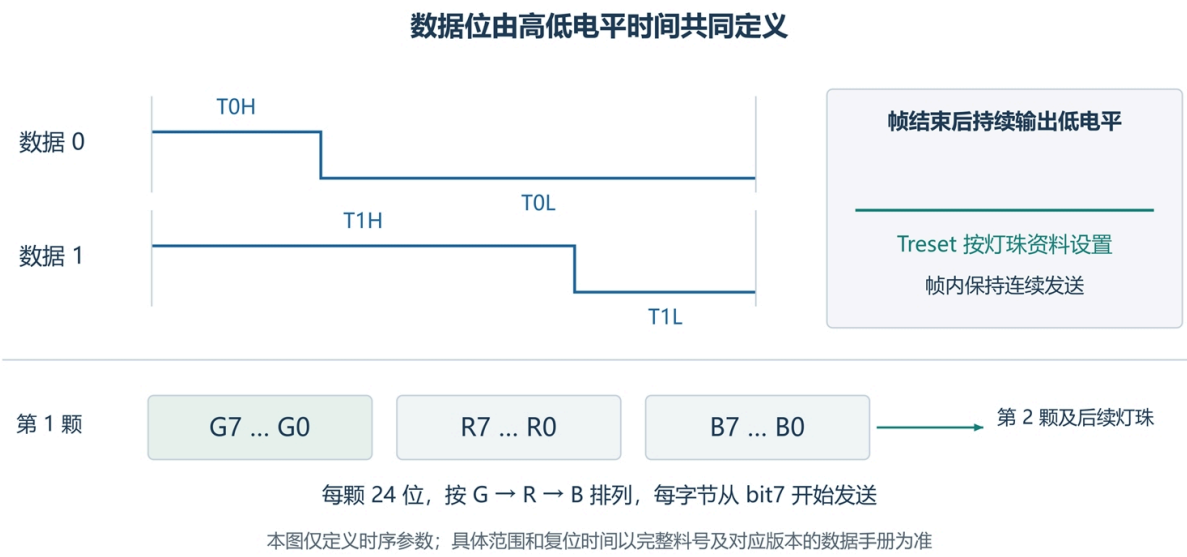
در این حالت INIT نقش آستانهٔ مقایسه را دارد و دیگر دورهٔ شمارش را تغییر نمی‌دهد. هنگامی که COUNT>I باشد، سیگنال داخلی CYCLE برابر 1 می‌شود و در صورتی که پایه در اختیار PIOC باشد، جهت آن خروجی بوده و خروجی تایمر اجازه داده شده باشد، این سیگنال روی IO0 اعمال می‌شود. ضریب مطابق سطح بالا برابر است با:

$$Duty = (255 - I) / 256$$

بخش ۲

نمونهٔ راه‌اندازی نوار LED تک‌سیمهٔ WS2812

◆ ۲.۱ پنجره‌های زمانی پروتکل



شکل ۲-۱ تعریف بیت WS2812

برای گواهی درستیِ زمان‌بندی، باید T0H، T0L، T1H، T1L و سطح پایینِ ریستِ پایان فریم را جداگانه بازبینی کرد؛ تفاوت صفر و یک تنها در سهم سطح بالاست. WS2812B رایج ۲۴ بیتی برای هر عدد 3 بایت مصرف می‌کند که به ترتیب G، R و B چیده می‌شوند و در هر بایت نخست bit7 ارسال می‌شود. در زنجیره‌کردن چند عدد، داده‌ها به‌ترتیب برای عدد اول تا آخرین قرار می‌گیرند؛ هر عدد 24 بیتِ متعلق به خود را برمی‌دارد و باقیمانده را از DOUT به عدد بعدی منتقل می‌کند. برای مثال، 00h، 20h، 00h یعنی مؤلفهٔ سبز و آبی صفر و مؤلفهٔ سرخ 20h است. برای قطعه‌هایی مانند خانوادهٔ RGBW باید تعداد بایت و ترتیب رنگ جداگانه تأیید شود. پروتکل WS2812 ساده اما در زمان‌بندی سخت‌گیر است: فقط از دو نوع بیت صفر و یک ساخته می‌شود، تلورانس عرض بیت ±150ns است، حجم داده از کوچک به بزرگ رشد می‌کند و ارسال باید یک‌بار و پیوسته انجام شود. مثال محلی EXAM\PIOC\1_Wire هم‌زمان دو مسیر فریم کوتاه (SFR) و فریم بلند (بارِ منطقهٔ برنامه) را پیاده کرده است و می‌توان از آن برای توضیح دو شیوهٔ پیاده‌سازی یک پروتکل بهره گرفت.

◆ ۲.۲ مؤلفه‌های پروژه و مقداردهی اولیه

جدول ۲-۱ فایل‌های اصلی پروژهٔ نمونه

فایل	تمرکز خواندن یا تغییر
Asm/RGB1W.ASM	تخصیص فرمان، واکشی از SFR و RAM، زمان‌بندی بیت، پایان فریم و بازگرداندن نتیجه
Asm/PIOC_INC.ASM	رجیسترها، شمارهٔ بیت‌ها و پارامتر دستورهای اختصاصی در سمت PIOC
Asm/RGB1W.BAT	فراخوانی اسمبلر WASM53B و تبدیل آرایهٔ C با BIN_HEX
User/RGB1W.c و RGB1W.h	مقداردهی اولیهٔ میزبان، آرایهٔ برنامه، آدرس بافر و رابط ارسال
Ld/Link.ld	دامنهٔ RAM میزبان و جداسازی SRAM مشترک با PIOC

تابع RGB1W_Init به‌ترتیب ساعت‌های AFIO، GPIOC و PIOC را روشن می‌کند، PC18/PC19 یا PC7 بازنگاشت‌شده را انتخاب و چندبرگرشدن پایه‌ها را تنظیم می‌کند؛ سپس PIOC را در حالت ریست نگه می‌دارد و با CLK_GATE=0، آرایهٔ PIOC_1W_CODE را به PIOC_SRAM_BASE کپی می‌کند. همین تابع بارگذاری را کامل می‌کند و رابط ارسالِ واقعی، بعدها ریست را آزاد کرده، ساعت PIOC را روشن و وظیفه را ثبت می‌کند.

اسکریپت لینکر میزبان، RAM معمولی را به 16KB آغازشده از 0x20000000 محدود می‌کند و منطقهٔ اشتراکی PIOC از 0x20004000 آغاز می‌شود و در مجموع 4KB است. هنگام پورت‌کردن، این جداسازی را حفظ کنید تا پشته، هیپ یا متغیرهای سراسری میزبان وارد منطقهٔ برنامهٔ PIOC نشوند. با استفاده از PC18/PC19 باید رابطهٔ چندبرگرشدن با رابط اشکال‌زدایی نیز مدیریت شود و اگر فقط از IO0 بازنگاشت‌شده استفاده می‌کنید، پیکربندی IO_EN را در همهٔ توابع ارسال هم‌زمان بازبینی کنید.

◆ ۲.۳ کدهای فرمان و کدهای نتیجه

میزبان نخست داده‌ها و پارامترها را آماده می‌کند و در پایان با نوشتن CTRL_WR فرمان را ثبت می‌کند. فرمان‌های زیر دامنهٔ رابطِ توافق‌شده در مثال‌اند و به آن معنا نیست که هر مقداری که می‌تواند به شاخه‌های اسمبلی برسد، برای فراخوانی مناسب است. مسیر RAM تعداد کل بایت‌ها را در DATA_REG0/1 و انتخاب کانال را در DATA_REG2 bit0 می‌گذارد؛ مسیر SFR اما بایت‌های آمادهٔ ارسال را در DATA_REG0~31 نگه می‌دارد.

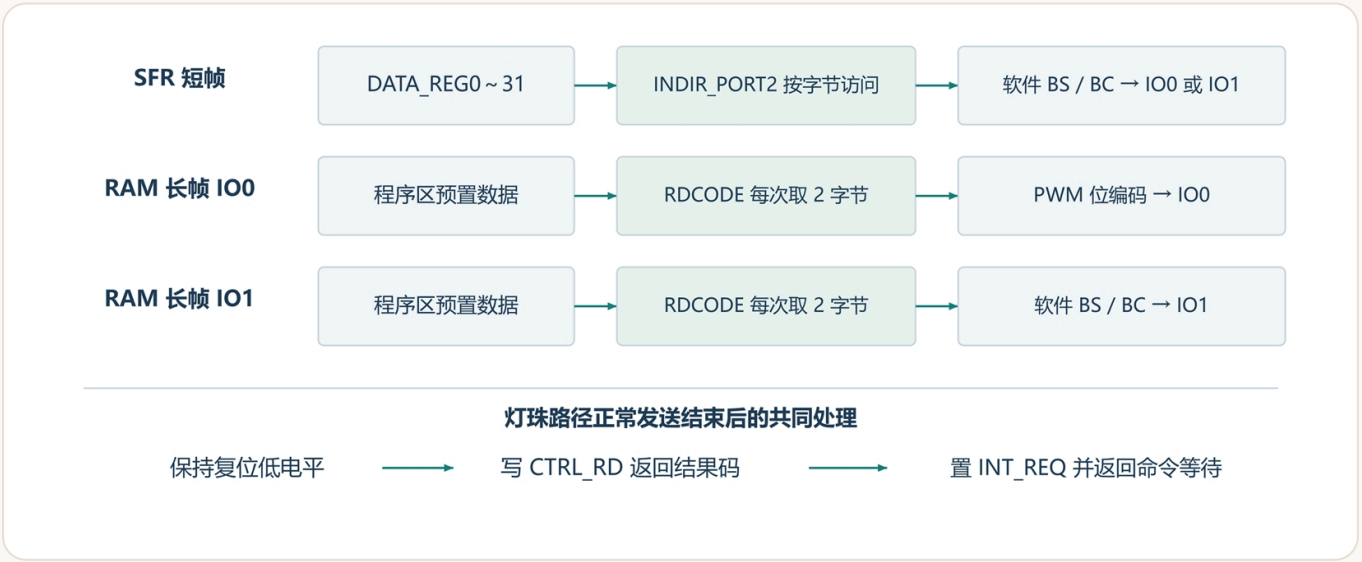
جدول ۲-۲ تخصیص کدهای فرمان در مثال

فرمان	معنا	منبع داده	کانال
0x01~0x20	فریم کوتاه؛ 7 بیت پایین تعداد بایت (1~32)	SFR از DATA_REG0 به بعد	IO0
0x44~0x60	فریم کوتاه؛ 6 بیت پایین تعداد بایت	SFR از DATA_REG0 به بعد	IO1
0x88~0xFC	فریم بلند؛ 7 بیت پایین عرض بیت N	بارِ منطقهٔ برنامه (RDCODE)	با DATA_REG2 انتخاب می‌شود
0x41~0x43	DS18B20	—	IO0

رابط فریم کوتهٔ IO1 طول را با 0x40 ترکیب و فرمان می‌سازد. طول‌های 1~3 متناظر با 0x41~0x43 هستند و پیش از هر چیز وارد شاخهٔ سنسور دما می‌شوند؛ بنابراین 3 بایت دادهٔ یک LED RGB را نمی‌توان مستقیماً با همین فرمان خامِ فریم کوتهٔ IO1 ارسال کرد. باید فرمانِ متداخل را دوباره تخصیص دهید و C و ASM را هم‌زمان اصلاح کنید، یا از مسیر ارسال دیگری که قبلاً اصلاح و راستی‌آزمایی شده استفاده نمایید. برای زنجیرهٔ کامل LEDهای RGB، تعداد بایت‌ها نیز باید مضرب 3 باشد.

در پایان وظیفه، PIOC نخست CTRL_RD را می‌نویسد، سپس INT_REQ را یک کرده و به WAIT_COMMAND بازمی‌گردد. رابط منتظرِ میزبان از طریق پرس‌وجوی INT_REQ پایان را تشخیص می‌دهد و با خواندن CTRL_RD نتیجه را می‌گیرد؛ پس از آن PIOC با دیدن SW_MR=0، INT_REQ را پاک می‌کند. اگر به شیوهٔ وقفه تغییر رویه دهید، باید برنامهٔ سرویس وقفه به‌طور یکپارچه نتیجه را بخواند و لایهٔ کاربرد را مطلع سازد تا کد پرس‌وجو و وقفه، یک وضعیتِ پایان را دو بار نخوانند.

◆ ۲.۴ مسیرهای فریم کوتاه و فریم بلند



شکل ۲-۲ سه مسیر در کد

مسیر SFR در هر نوبت بیشترین 32 بایت منتقل می‌کند و دادهٔ 10 LED کامل RGB را در خود جای می‌دهد. در نقطهٔ ورود، کدگذاری بیت بسته می‌شود، مقدار قفل OUT متناظر روی سطح پایین تنظیم و جهت خروجی پیکربندی می‌شود و سپس بررسی می‌شود که ورودی پایین است یا نه. پس از آن با INDIR_ADDR تعداد بایت‌های باقی‌مانده نگه داشته می‌شود و با MOVIA، گروه دوم آدرس‌های غیرمستقیم یعنی INDIR_ADDR2 به DATA_REG0 اشاره می‌کند.

```
        BS    SFR_PORT_IO,SB_PORT_OUT0   ; لبهٔ بالارونده
        CALL  RGB1W_DLY_0H               ; سطح بالا 292 ns
        BTSS  SFR_INDIR_PORT2,7          ; اگر بیت داده 1 است، پرش
        BC    SFR_PORT_IO,SB_PORT_OUT0   ; دادهٔ 0: پایین‌کشیدن زودتر
        CALL  RGB1W_DLY_HALF             ; پُرکردن تا نیمهٔ دوره
        BC    SFR_PORT_IO,SB_PORT_OUT0   ; دادهٔ 1: پایین‌کشیدن در اینجا
        CALL  RGB1W_DLY_1L               ; سطح پایین 292 ns
        ; ... bit6~bit0 برای جبران پیشروی _B0 به همان شکل باز می‌شوند؛ بیت آخر از واریانت
اشاره‌گر استفاده می‌کند
```

این قطعه کد یک بیت دادهٔ معمولی را پردازش می‌کند. برای دادهٔ 0، BTSS از BC اول عبور نمی‌کند و خروجی زودتر پایین کشیده می‌شود؛ برای دادهٔ 1 همان BC پرش می‌شود و سطح بالا تا BC دوم حفظ می‌شود. DLY_HALF برای پُرکردن بازهٔ میانی است و به آن معنا نیست که سیگنال را در مرکز دورهٔ بیت پایین بکشد. هر دو مسیر داده باید سربارِ پرش یا نشدن BTSS و نیز CALL/RET را در محاسبهٔ زمان داشته باشند.



شکل ۲-۳ زمان‌بندی بیت نرم‌افزاری

DLY_0H هشت دستور NOP به‌همراه RET است و در مجموع 10 چرخه؛ DLY_1L 11 چرخه؛ و DLY_HALF با در نظر گرفتن فراخوانی تو در تو و مسیر اجرای ترتیبی تا DLY_1L، بدنهٔ 25 چرخه‌ای دارد. با لحاظ فراخوانی و انشعاب، برای بیت معمولی T0H/T0L برابر 14/42 چرخه و T1H/T1L برابر 42/14 چرخه می‌شود. در 48MHz این‌ها به‌ترتیب حدود 0.292/0.875us و 0.875/0.292us می‌شوند و دورهٔ بیت حدود 1.167us است.

بررسی بیت‌های bit7~bit1 نشانگر داده را به‌طور خودکار جلو نمی‌برد؛ در انتهای بایت، MOV SFR_INDIR_PORT2,A افزایش خودکار INDIR_ADDR2 را راه می‌اندازد و سپس شمارندهٔ بایت کم می‌شود. bit0 از نقطهٔ ورود تأخیرِ _B0 برای

جبران سربار واکشی و حلقه بهره می‌گیرد. هنگام تغییر، نمی‌توان فقط NOPهای بیت معمولی را تنظیم کرد؛ باید بیت پایانی بایت، بایت‌های پیاپی و مسیر جمع‌بندی پس از آخرین بیت را نیز هم‌زمان حساب کرد.

◆ واکشی داده و کدگذاری بیت در فریم بلند RAM

مسیر RAM داده‌ها را از پیش در منطقهٔ برنامهٔ PIOC می‌گذارد و با کمک RDCODE در هر نوبت یک کلمهٔ 16 بیتی می‌خواند. بایت پایین در A و بایت بالا در INDIR_ADDR قرار می‌گیرد؛ پس نخست بایت پایین و سپس بایت بالا ارسال می‌شود و درون هر بایت همچنان ارسال از bit7 تا bit0 است. VAR_ADDR_L/H آدرس واکشی بعدی را جداگانه نگه می‌دارد، VAR_SIZE_L/H تعداد بایت‌های باقی‌مانده را ثبت می‌کند و DATA_EXCH بایت جاری را موقتاً نگه می‌دارد.

◆ IO0 از کدگذاری بیت سخت‌افزاری استفاده می‌کند

شاخهٔ IO0 مقدار BIT_CONFIG را روی 0x80 می‌گذارد؛ یعنی TX_EN=1 و CODE_MOD=0. 7 بیت پایین فرمان به‌عنوان میدان دوره به کار می‌رود: در 48MHz مقدار 56 متناظر با فرمان 0xB8 و در 24MHz مقدار 28 متناظر با فرمان 0x9C است و عرض بیتِ اسمی مثال در هر دو حدود 1.167us است. قطعهٔ زیر آدرس خواندن اصلیِ منطقهٔ برنامه را حفظ می‌کند.

```
        CLR   VAR_ADDR_L
        CLR   SFR_INDIR_ADDR
        MOVL  0X02                ; بایت بالای آدرس کلمهٔ اصلی
        MOVA  VAR_ADDR_H
        RDCODE                    ; A=بایت پایین، INDIR_ADDR=بایت بالا
        MOVA  SFR_DATA_EXCH
        ANDL  0X80                ; بایت نخست bit7
        IOR   SFR_INDIR_ADDR2,A   ; تلفیق میدان دوره
        NOP
        MOVA  SFR_BIT_CYCLE       ; ارسال فعال شده: بارگذاری بیت اول و آغاز
RGB1W_WORD_NEXT:
        WAITB WB_PORT_I0_RISE     ; تأیید آغاز بیت جاری
        BP2F  BO_BIT_TX_O0,6      ; پیش‌بارگذاری بیت بعدی
```

لبهٔ بالارونده در این‌جا آغاز بیتِ کدگذاری‌شدهٔ PWM جاری را نشان می‌دهد، نه پایان ارسال بیت جاری. TX_O0 ساختار دوبافر دارد و برنامه پس از آغاز بیت جاری، بیت بعدی را بارگذاری می‌کند؛ بیت جاری همچنان توسط کدگذار ادامه می‌یابد. WAITB روی سجل تشخیص لبهٔ بالارونده انتظار می‌کشد و هنگام خروج آن را پاک می‌کند؛ نوشتن BIT_CYCLE نیز سجل لبهٔ IO0 را پاک می‌کند، پس ترتیب «آغاز بیت نخست، پاک‌شدن سجل و نوشتن بیت‌های بعدی» را نمی‌توان دل‌بخواه جابه‌جا کرد.

برای کم‌کردن یک واحد از طول، کد نخست DEC VAR_SIZE_L و سپس INC VAR_SIZE_L,A را اجرا می‌کند. اگر بایت پایین اصلی 00h باشد، پس از یک واحد کم‌شدن FFh می‌شود؛ سپس با افزودن 1 به A مقدار 00h حاصل و پرچم Z برقرار می‌شود و در این هنگام از بایت بالا یک واحد کم می‌شود. نتیجهٔ دستور دوم در A نوشته می‌شود و مقدارِ کاسته‌شدهٔ VAR_SIZE_L را به حالت اول بازنمی‌گرداند. شناخت رجیستر مقصد کمک می‌کند هنگام تغییر منطق بررسی طول، پردازش قرضِ 16 بیتی از کار نیفتد.

```
RGB1W_BIT_LAST:
        BP2F   BO_BIT_TX_O0,0
        WAITB WB_PORT_I0_RISE      ; آغاز بیت پایانی
        BC     SFR_BIT_CYCLE,SB_BIT_TX_O0
        WAITB WB_PORT_I0_FALL      ; پایان سطح بالای بیت پایانی
        CLR    SFR_BIT_CYCLE       ; توقف کدگذاری
        JMP    RGB1W_LOAD
```

پس از بارگذاری بیت پایانی، کد نخست لبهٔ بالارونده‌ٔ همان بیت را انتظار می‌کشد، سپس TX_O0 را پاک کرده و لبهٔ پایین‌رونده را انتظار می‌کشد؛ بعد کدگذاری را متوقف می‌کند و وارد مرحلهٔ سطح پایین ریست می‌شود. BC BIT_CYCLE نیز سجل لبهٔ قدیمی را پاک می‌کند و در این‌جا برای آن به کار می‌رود که لبهٔ پایین‌روندهٔ پیشین به‌اشتباه پایان بیت آخر تلقی نشود. نمی‌توان بلافاصله پس از نوشتن بیت پایانی، کدگذار را بست.

◆ IO1 همچنان از خروجی نرم‌افزاری استفاده می‌کند

RGB1W_LONG_1 اگرچه همانند مسیر قبلی از منطقهٔ برنامه واکشی می‌کند، اما IO1 قابلیت خروجی کدگذاری بیتِ IO0 را ندارد؛ بنابراین شکل موج با BS/BC و زیربرنامه‌های تأخیر ساخته می‌شود. میدان دوره در فرمان نمی‌تواند پهنای پالس نرم‌افزاریِ این مسیر را مستقیماً تغییر دهد.

◆ ۲.۵ فراخوانی میزبان و یک ارسال کامل

برای اولین راه‌اندازی می‌توان از مسیر SFR روی IO0 شروع کرد. پس از آماده‌سازی ساعت سیستم، سیم‌کشی، سطوح منطقی و سطح پایینِ ریست فریم نخست، فراخوانی زیر دادهٔ سرخ با درخشندگی کم را به یک LED می‌فرستد؛ فایل‌های سرآیند مثال و پیکربندی لینک باید به پروژهٔ میزبان افزوده شده باشند.

جدول ۲-۳ ترتیب عملیات دو رابط ارسال میزبان

مرحله	مسیر SFR
آماده‌سازی داده	چیدن بایت‌های GRB در بافر میزبان
ریست و کپی	پس از ریست، نخست PIOC راه می‌افتد و سپس کپی به DATA_REG
ثبت وظیفه	در پایان، فرمانِ مرکب از طول و کانال نوشته می‌شود
خواندن نتیجه	انتظار برای INT_REQ و سپس خواندن CTRL_RD
فریم بعدی	پردازش نتیجه و اطمینان از امکان ثبت دوباره

◆ تنظیم زمان‌بندی و بازتولید فریم‌ور

اگر از خروجی نرم‌افزاری استفاده می‌کنید، می‌توانید نخست تعداد چرخهٔ هدف هر بخش را تعیین کنید و سپس پیاده‌سازی اسمبلی را تغییر دهید. پس از اصلاح ASM، فایل RGB1W.BAT همراه را اجرا کنید؛ خطاها و هشدارهای فایل LST، آدرس

نمادها و انتهای کد را بررسی کنید؛ مطمئن شوید RGB1W_inc.h تولیدشدهٔ تبدیل BIN از نظر طول و محتوای بایت‌ها با BIN یکسان است و سپس آرایهٔ PIOC_1W_CODE را در RGB1W.c به‌روزرسانی کنید. در پایان، پروژهٔ میزبان را دوباره ترجمه و بارگذاری کنید تا تابع مقداردهی اولیه، برنامهٔ جدید PIOC را بارگذاری کند. تنها اصلاح ASM یا تنها ترجمهٔ دوبارهٔ پروژهٔ میزبان، هیچ‌کدام تضمین نمی‌کند که آخرین کد ماشین PIOC در حال اجراست.

اگر نقطهٔ آغاز RAM دست‌خوش تغییر می‌شود، ماکروی C و دو آدرس کلمه در ASM باید هم‌زمان اصلاح شوند؛ اگر اصلاح 24MHz در میان است، باید آدرس برچسبِ وصله را از LST جاری تأیید و به آفست بایت تبدیل کنید. آدرس برنامهٔ PIOC بر حسب کلمه و اشاره‌گر C بر حسب بایت است و رابطهٔ تبدیل این دو چنین است:

$$\mathrm{Offset}_{\mathrm{byte}} = 2 * \mathrm{Addr}_{\mathrm{word}}$$

پیوست

تاریخچهٔ نسخه‌ها و بیانیه

جدول تاریخچه — تاریخچهٔ نسخه‌های سند اصلی AN23000

تاریخ	نسخه	شرح تغییرات
2026/11/02	V1.0	انتشار نسخهٔ اول

بیانیه

این راهنما صرفاً جهت مرجع ارائه شده است و نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند. ترجمهٔ فارسی این سند توسط تیم معمار الکترونیک صنعت تهیه شده است؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول واژگان — واژگان کلیدی به‌کاررفته در این جزوه

واژه	یعنی چه؟
PIOC	واحد جانبی (Peripheral) قابل‌برنامه‌ریزی ورودی/خروجی روی سری‌های CH32X035، CH32H417 و CH32V205؛ موج پروتکل را روی دو پایه و با چرخه‌های دستور معین تولید می‌کند

واژه	یعنی چه؟
eMCU	نام دیگر PIOC در راهنمای تراشه؛ یک میکروکنترلر کوچکِ تعبیه‌شده در کنار هستهٔ اصلی
RISC8B	هستهٔ ۸بیتی PIOC با دادهٔ 8 بیت، دستور 16 بیتی، 66 دستور و اجرای تک‌چرخه‌ای
SFR	رجیسترهای عملکرد خاص؛ پنجرهٔ ارتباط مستقیم میزبان و PIOC برای فرمان، پارامتر و داده — 49 عدد شامل 33 بایت رجیستر داده
CTRL_WR / CTRL_RD	رجیستر نوشتن فرمان میزبان به PIOC / رجیستر فقط‌خواندنی نتیجه؛ نوشتن جعلی روی CTRL_RD راه پاک‌کردن INT_REQ است
MW_SR / SW_MR	بیت «فرمان برداشته شد» (پس از نوشتن میزبان یک می‌شود) / بیت «نتیجه خوانده شد» (پس از نوشتن نتیجه توسط PIOC یک می‌شود)
INT_REQ	بیت درخواست وقفه از طرف PIOC به میزبان؛ پس از برداشتن نتیجه، فریم‌ور آن را پاک می‌کند
CLK_GATE	گیت ساعتِ فضای 4KB مشترک؛ 0: SRAM میزبان، 1: ROM برنامهٔ PIOC — نوشتن فقط با گیت بسته مجاز است
RDCODE	دستور ویژهٔ خواندن منطقهٔ برنامه توسط PIOC؛ با آدرس کلمهٔ 11 بیتی، هر بار یک کلمهٔ 16 بیتی برمی‌گرداند و 3 چرخه طول می‌کشد
BIT_CYCLE	رجیستر راه‌اندازی دورهٔ بیت؛ نوشتن بیت داده در bit7 یک دورهٔ بیت کامل تولید می‌کند و نوشتن در آن سجل لبهٔ IO0 را نیز پاک می‌کند
WAITB	دستور انتظار بر هشت رویداد همگام‌سازی: فرمان میزبان، خواندن نتیجه، TAIL، لبه‌های IO0 و مقایسهٔ ورودی/خروجی
WS2812	LED تک‌سیمهٔ RGB با زمان‌بندی سخت‌گیرانه؛ هر عدد 24 بیت با ترتیب GRB می‌گیرد و تلورانس عرض بیت آن ±150ns است
DS18B20	سنسور دمای تک‌سیمه؛ در مثال، فرمان‌های 0x43~0x41 مسیر IO1 به شاخهٔ آن می‌رسند



معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

با تشکر، حسن معماری

ترجمهٔ فارسی سند AN23000 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۲۳

راه‌های ارتباط با ما

گیت‌هاب	https://github.com/MEMAR-ELECTRONIC-SANAT
کانال تلگرام	https://t.me/Memar_Electronic
پیج اینستاگرام	https://www.instagram.com/Memar.Electronic
ایمیل	Memari.h@gmail.com